

https://chatgpt.com/s/_6a34deb460c08191ad2b49fd8f0d5ae5

Let's build it in a way that you can:

1. Put it on your resume
2. Explain it confidently in interviews
3. Upload it to GitHub
4. Extend it later with Streamlit and chat history

Project: Enterprise Knowledge Assistant (RAG)

What are we building?

Imagine a company uploads:

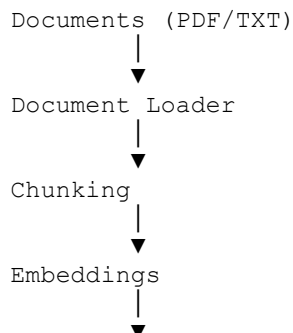
- HR Policy PDF
- Leave Policy PDF
- Employee Handbook PDF
- IT Security Policy PDF

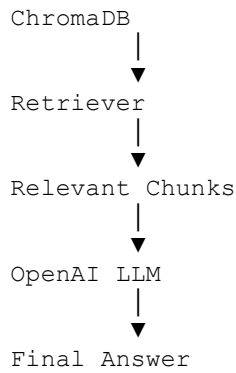
User asks:

"How many casual leaves do employees get?"

Instead of searching manually, the chatbot answers directly from company documents.

Architecture





This is exactly what interviewers expect when they ask:

Explain a RAG pipeline.

Phase 1: Environment Setup

Create project:

```
mkdir Enterprise-Knowledge-Assistant
cd Enterprise-Knowledge-Assistant
```

Create virtual environment:

```
python -m venv .venv
```

Activate:

```
.venv\Scripts\activate
```

Install packages:

```
pip install langchain
pip install langchain-community
pip install langchain-openai
pip install chromadb
pip install pypdf
pip install sentence-transformers
pip install python-dotenv
pip install streamlit
```

Project Structure

Enterprise-Knowledge-Assistant/

```
|
└─ data/
```

```
|   | hr_policy.pdf
|   | leave_policy.pdf
|--- vectordb/
|   | src/
|   | | load_documents.py
|   | | create_embeddings.py
|   | | chatbot.py
|--- .env
|--- requirements.txt
|--- app.py
```

Phase 2: Load Documents

load_documents.py

```
from langchain_community.document_loaders import PyPDFLoader
from langchain_community.document_loaders import DirectoryLoader

loader = DirectoryLoader(
    "../data",
    glob="*.pdf",
    loader_cls=PyPDFLoader
)

documents = loader.load()

print(len(documents))
```

Phase 3: Chunking

Use the same splitter you're currently learning.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)

chunks = splitter.split_documents(documents)

print(len(chunks))
```

Interview Question:

Why chunk_overlap?

Answer:

To preserve context between adjacent chunks and reduce information loss.

Phase 4: Embeddings

create_embeddings.py

```
from langchain_openai import OpenAIEmbeddings
```

OR

Free option:

```
from langchain_community.embeddings import HuggingFaceEmbeddings
```

Use:

```
embeddings = HuggingFaceEmbeddings(  
    model_name="sentence-transformers/all-MiniLM-L6-v2"  
)
```

Phase 5: Store in ChromaDB

```
from langchain_community.vectorstores import Chroma  
  
vectorstore = Chroma.from_documents(  
    documents=chunks,  
    embedding=embeddings,  
    persist_directory="../vectordb"  
)  
  
vectorstore.persist()
```

Now all chunks are stored as vectors.

Phase 6: Semantic Search

Load vector DB:

```
vectorstore = Chroma(  
    persist_directory="../vectordb",  
    embedding_function=embeddings  
)
```

Search:

```
results = vectorstore.similarity_search(
    "How many casual leaves are allowed?",
    k=3
)
```

Interview:

What happens here?

Answer:

Query is converted into an embedding and compared against stored document embeddings using vector similarity.

Phase 7: LLM Integration

Create .env

```
OPENAI_API_KEY=YOUR_KEY
```

Load:

```
from dotenv import load_dotenv
import os

load_dotenv()

api_key = os.getenv("OPENAI_API_KEY")
```

LLM:

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0
)
```

Phase 8: Create Retriever

```
retriever = vectorstore.as_retriever(
    search_kwargs={"k": 3}
)
```

Retrieve context:

```
docs = retriever.get_relevant_documents(question)
```

Phase 9: Prompt Engineering

```
prompt = f"""
You are an enterprise knowledge assistant.

Answer only from the provided context.

If information is not available,
say:
"I couldn't find this information in the documents."

Context:
{context}

Question:
{question}
"""
```

This prevents hallucination.

Phase 10: Generate Final Answer

```
response = llm.invoke(prompt)

print(response.content)
```

Phase 11: Build Chatbot

chatbot.py

```
while True:

    question = input("Ask Question: ")

    if question.lower() == "exit":
        break

    docs = retriever.get_relevant_documents(question)

    context = "\n".join(
        [doc.page_content for doc in docs]
    )

    prompt = f"""
Context:
{context}

Question:
{question}
"""
```

```
response = llm.invoke(prompt)

print(response.content)
```

Phase 12: Streamlit UI

app.py

```
import streamlit as st

st.title("Enterprise Knowledge Assistant")

question = st.text_input(
    "Ask your question"
)

if question:
    response = ask_question(question)
    st.write(response)
```

Run:

```
streamlit run app.py
```

Resume-Worthy Enhancements

After the basic version works, add:

1. Conversational Memory

ConversationBufferMemory

2. Source Citations

Show:

Answer:
...

Source:
HR_Policy.pdf Page 12

3. Multi-PDF Upload

Upload PDFs dynamically.

4. Hybrid Search

Combine:

Vector Search
+
Keyword Search

5. Conversation History

Store previous questions.

6. Evaluation Metrics

Measure:

- Retrieval Accuracy
- Context Relevance
- Groundedness

How to Explain in an Interview

I built an Enterprise Knowledge Assistant using LangChain, ChromaDB, OpenAI APIs, and RAG architecture. Documents were loaded and chunked using RecursiveCharacterTextSplitter. Chunks were converted into embeddings using sentence-transformers and stored in ChromaDB. User queries were semantically matched against document embeddings, and the most relevant chunks were retrieved and passed to an LLM through a carefully engineered prompt. The solution generated grounded responses based only on enterprise documents and reduced hallucinations through contextual retrieval.

This is strong enough for a **1–2 year GenAI Engineer interview**.

My suggestion: since you're already following the RAG tutorial and have successfully reached the chunking stage, next we should build this project **step-by-step on your machine**, starting with **document loading** → **chunking** → **embeddings** → **ChromaDB**, and I'll help you write and run every file.